Student: Paul Runov

Course: ECE 5210

Subject: IIR Data Filtering

Date: April 15, 2026

1 Introduction

This chapter explores the intricate process of Infinite Impulse Response (IIR) filtering and its application in sound restoration, with a specific focus on designing and implementing filters on the STM32F769I microcontroller. This study is inspired by the pioneering work of Thomas Stockham in audio digital signal processing (DSP) during the late 1960s and 1970s at the University of Utah. The goal is to develop a robust method for removing high-frequency noise from audio recordings, enhancing their clarity and listenability. The exercise involves practical DSP techniques and tests theoretical knowledge against real-world signal processing challenges.

2 Theory

2.1 Theory: IIR Filter Code Implementation

The implementation of Infinite Impulse Response (IIR) filters on the STM32F769I board involves coding a routine that applies cascaded second-order sections (biquad stages) for filtering audio signals. Each biquad stage is a building block that processes the input signal using a pair of feedforward (b) and feedback (a) coefficients. This design enhances stability and minimizes the risk of numerical errors that are common in fixed-point arithmetic systems.

The routine, `iir_sos()`, processes each audio sample sequentially through multiple biquad stages, using the formula:

$$H(z) = \prod_{k=1}^{Ns} \frac{b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}}{1 + a_{1k}z^{-1} + a_{2k}z^{-2}}$$

Since the code for a simple IIR was created in lab 6, this can be reused as function for a second order section. For loops are quite slow, which is why the SOS is hard coded as multiplications. Then a different function is created to loop over all of the second order sections.

3 Filter Design and Coefficients

3.1 Design Methodology

The Infinite Impulse Response (IIR) filter for audio restoration was designed using the elliptical filter approach, which is ideal for applications requiring sharp cutoffs and minimal ripple in the passband. The design process involved defining the passband and stopband frequencies, as well as the tolerances for ripple and attenuation.

To meet the specific audio processing needs, the filter was configured with a passband cutoff at 2.4 kHz, ensuring that most of the speech content is preserved. The stopband was set to begin at 3 kHz, to attenuate high-frequency noise effectively. These cutoff frequencies were normalized relative to the Nyquist frequency, given the sampling rate of 48 kHz used in audio recording.

The passband ripple and stopband attenuation were chosen based on desired specifications for audio clarity and noise suppression. A ripple of 0.1 (in linear scale) was allowed in the passband, which provides a good balance between attenuation and the preservation of the audio's natural qualities. The stopband attenuation was set to 0.001 (in linear scale), corresponding to a stringent requirement that ensures substantial reduction of unwanted high-frequency components.

Using these parameters, the order of the filter and the critical frequencies were calculated to achieve the optimal balance between efficiency and performance. The elliptical filter design was chosen for its effectiveness in achieving sharp roll-off and deep stopbands with relatively lower order compared to other filter types like Butterworth or Chebyshev.

The coefficients for the filter were determined to provide the necessary frequency response characteristics. These coefficients are critical as they define the filter's behavior across the frequency spectrum, impacting how well the filter meets its design goals. The elliptical filter's ability to control both ripple and roll-off steepness makes it particularly suitable for applications like audio restoration, where both the clarity of speech and the suppression of noise are paramount.

4 Results

4.1 Questions

Question: Describe the difference in audio quality between the right channel (noisy) and the left channel (filtered). Was the filter effective in improving audio clarity?
Response: The filter did a decent job at filtering out the noise. Granted, this noise seemed very superficial. I actually had to turn off the passthrough because it was so bad in the headphones to properly identify the speech.

Question: Identify the speaker, the location, and the date of the speech in the filtered audio.
Response: The speech was "We choose to go to the moon" by JFK. It was said on September 12, 1962 at Rice University

Question: How many biquad stages were you able to implement and test on the STM32F769I board, and how did this affect the filter's performance?
Response: Initially I was only able to get 5 filter stages. Then something happened and my filter broke. I ended up rewriting my code and instead of using for loops, I manually hard coded the multiplies. This increased the number of section to 15.

4.2 Plots

The poles and zeros found for the elliptical filter that was created in lab, and the results plotted in Figure 1. It is seen that the Zeros all lie on the unit circle, and the poles are also close to the unit circle and X axis.

The frequency response was plotted in Figure 2. the magnitude lines up almost perfectly between the measure and designed filter. The phase lines up pretty close. There is a little vertical jump, but that is due to using only 120 points to plot the data. If more points were used, it would be smoother.

5 Discussion and Conclusions

This lab was fairly easy, until my code broke. I got my code working during the first lab period, and even ended up helping some classmates get their code working also. Somehow my code decided to just shit itself later, and I couldn't figure out why. After extensive testing, I realized I wasn't getting more than 1 stage on the hardware (even though I measured 5 in lab). Because of this, I ended up rewriting the IIR section by hard-coding all of the calculations. This allowed my to get 15 sections out of the board.

Overall this lab was a good way to remind me I still suck at coding. It also taught me how to optimize code to make it faster to run on a board. It also solidified the concept of creating filters to a specified tolerance using python tools.

6 Plots

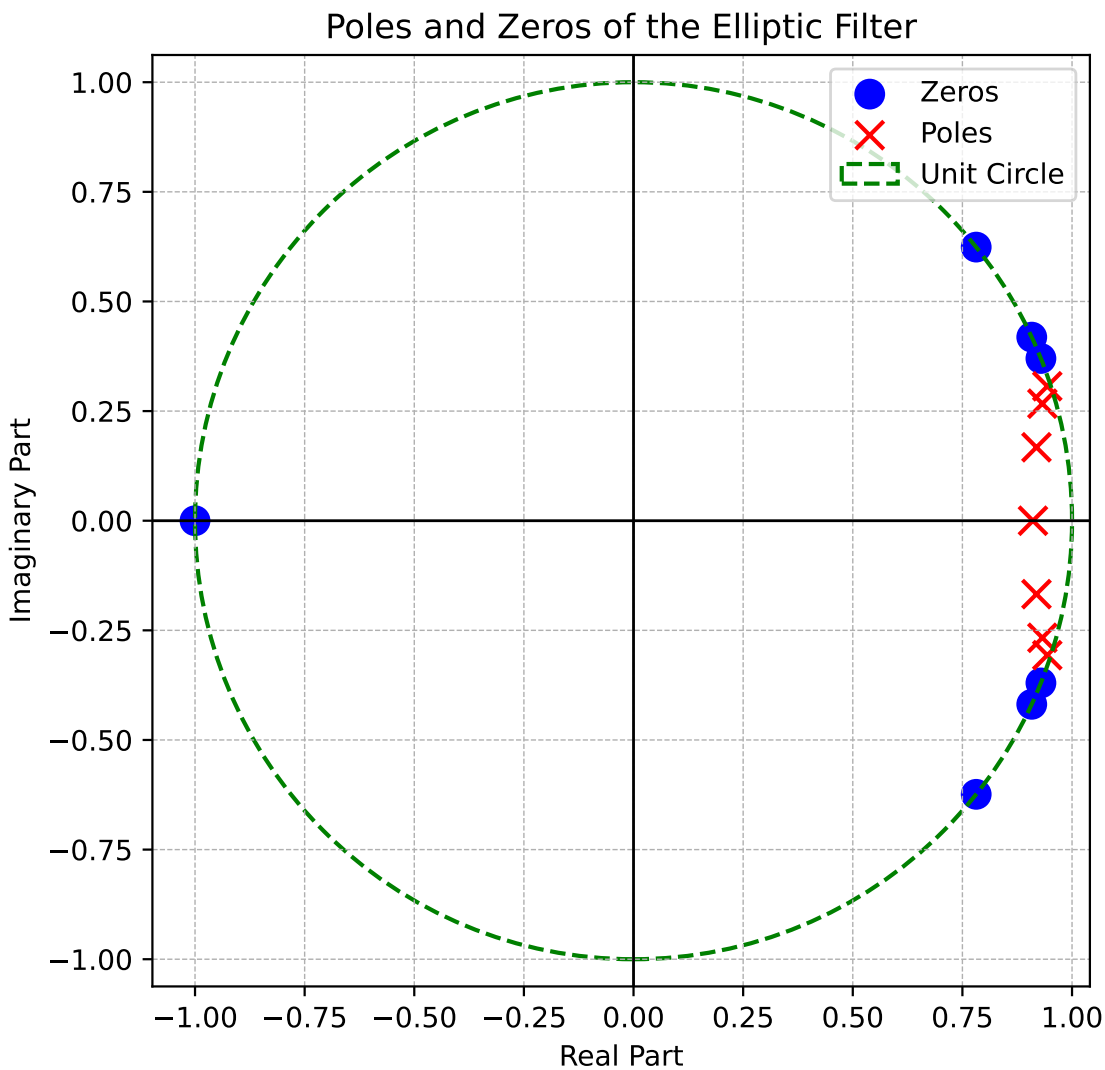

Figure 1: Poles and Zeros of the system

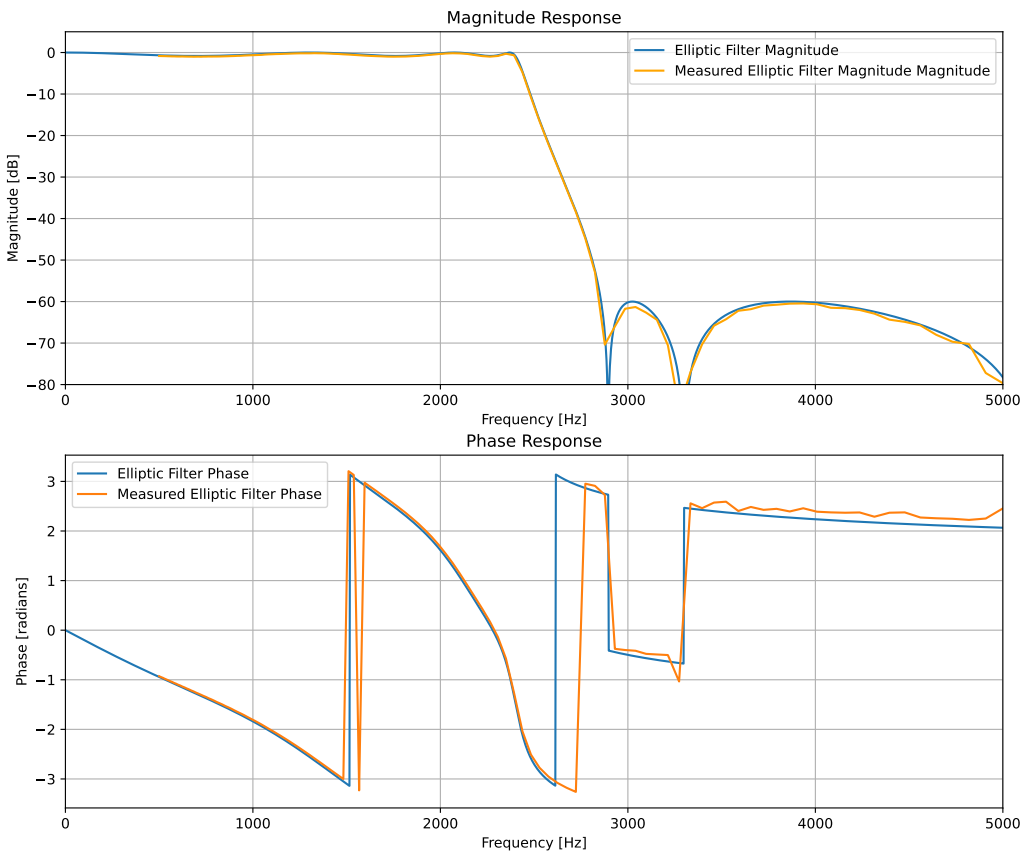

Figure 2: Frequency Response of the Systems